KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Codebase Q&A Assistant

A Retrieval-Augmented Generation (RAG)

System for Understanding Software Repositories

Project ID: T053

Team ID : PRJ063

Team GitHub Repository: https://github.com/harshavardhanreddyundela-bot/codebase_q-a.git

Student Name	Registration Number	Individual GitHub Link
UNDELA HARSHAVARDHAN REDDY	99230040180	https://github.com/harshavardhanreddyundela-bot/ASSIGNMENTS.git
KUNDURU SNEHALATHA REDDY	99230040357	https://github.com/Sneh-04/ASSIGNMENTS
KUPPALA KIRAN	99230041193	https://github.com/kirankuppala28/GEN_AI_ASSIGNMENT.git
ALLU SURESH	99230041244	https://github.com/suresh/Gen_Ai_Assignment.git
KASA VISHNU VARDHAN	99230041269	https://github.com/vishnu-0422

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

Section	Topic	Reference
1.	Introduction	Section 1
2.	Problem Statement & Objectives	Section 2
3.	System Architecture (RAG Pipeline)	Section 3
4.	Implementation Details & Tech Stack	Section 4
5.	Results and Testing	Section 5
6.	Conclusion & Future Scope	Section 6

1. Introduction

Large Language Models (LLMs) can explain programming concepts and generate code, but they do not automatically have reliable, continuous knowledge of every private or frequently changing software repository. Developers working with unfamiliar or large codebases may need to inspect many files to understand where a feature is implemented and how different modules interact.

This project proposes a Codebase Q&A Assistant that uses Retrieval-Augmented Generation (RAG) to answer natural-language questions about a selected software repository. It indexes source files, retrieves relevant code snippets for a user query, and supplies that context to an LLM to generate an explanation with source-file references.

2. Problem Statement & Objectives

Problem: Understanding an unfamiliar codebase can be time-consuming. Developers may manually search folders, inspect functions and classes, follow imports, and trace how modules communicate. General-purpose chat tools can help, but answers depend on the code and context supplied or made accessible to them. Without repository-specific retrieval and source references, it can be difficult to verify an answer against the actual project.

Objectives:

Develop an assistant that answers questions using the contents of a selected software repository.

Ingest and preprocess supported source-code and project documentation files.

Split code into meaningful chunks and generate vector embeddings.

Store embeddings with metadata such as file path and chunk information in a vector database.

Retrieve relevant code context for each question and pass it to an LLM.

Display answers with source-file references to help users verify the response.

3. System Architecture (RAG Pipeline)

The system is organized into two primary phases: Repository Ingestion and Query Processing. The ingestion phase prepares a searchable representation of the codebase. The query phase retrieves relevant code and uses it as context for answer generation.

3.1 Repository Ingestion Phase

The user provides a repository, for example through a ZIP upload or a GitHub repository URL. The application discovers supported files and excludes irrelevant or oversized content such as virtual environments, dependency folders, build outputs, binary files, and secrets. Source files and useful documentation are read and divided into chunks. Where supported, chunking can preserve function or class boundaries; otherwise, a text splitter can be used. Each chunk is converted into an embedding and stored in a vector database together with metadata such as repository name, file path, language, and chunk index.

3.2 Query & Generation Phase

When a user submits a question, the question is converted into an embedding using the same embedding model. The vector database performs similarity search and returns the most relevant code chunks. The application builds a prompt containing the question, retrieved context, and instructions to avoid unsupported claims. The LLM generates an answer based on the retrieved context. The interface presents the response and the source file paths (and line references where available).

RAG Pipeline Overview

Phase	Process
Repository ingestion	GitHub URL / ZIP upload → File filtering → Code & documentation loading
Indexing	Code-aware chunking → Embedding generation → Vector database
Query processing	User question → Query embedding → Similarity search / retrieval
Answer generation	Retrieved code context → LLM prompt → Grounded answer + sources

4. Implementation Details & Tech Stack

Component	Technology	Role
Language	Python 3.11	Core runtime
LLM Provider	Google Gemini (`gemini-3.5-flash-lite`)	Code reasoning & grounded answer generation
Embeddings	`gemini-embedding-001` + ONNX MiniLM	Semantic vector generation
Vector DB	ChromaDB (`chromadb`)	Persistent vector & metadata store
Retrieval	Hybrid (Vector Cosine + Keyword RRF)	Precision symbol & function retrieval
Code Parser	Python AST (`ast`) + Regex	Function/class boundary preservation
Security	Custom Regex Scanner + Sandboxing	Secret redaction & indirect injection protection
Web UI	Streamlit (`streamlit`)	Browser-based interactive dashboard

4.1 Sample Chunking Configuration

A text splitter can be used as a baseline for chunking source files. The values below are illustrative and should be tuned for the programming languages and repository size. For better code retrieval, function- or class-aware chunking can be added.

```
RecursiveCharacterTextSplitter(
chunk_size=1000,
chunk_overlap=200,
length_function=len
)
```

5. Results and Testing

The project should be evaluated using a representative set of questions about one or more repositories. Since measured results were not provided in the project brief, no accuracy percentage or response-time result is claimed in this document. Record actual measurements after running the application.

Test	Example Question	Expected Outcome
Feature location	Where is user authentication implemented?	Relevant authentication files and code sections are retrieved.
Function explanation	Explain the register_user() function.	Explanation reflects the retrieved function and its behavior.
Cross-file flow	How does a request reach the database layer?	Answer connects relevant route, service, and database files where available.
Dependency lookup	Which library is used for API routing?	Answer identifies dependency evidence from project files.
Unknown information	Where is payment processing implemented? (when absent)	Assistant states that supporting evidence was not found rather than inventing a file or function.

Evaluation measures: retrieval relevance (whether the correct files/chunks are returned), source correctness (whether cited paths support the answer), answer usefulness, handling of unsupported questions, and average response time. Report the dataset size, test procedure, and measured values in the final version.

6. Conclusion & Future Scope

Conclusion: The Codebase Q&A Assistant is designed to make unfamiliar software repositories easier to understand. By combining repository ingestion, code chunking, embeddings, vector similarity search, and LLM-based generation, it provides a focused question-answering workflow grounded in retrieved project files. Its purpose is not to replace general-purpose assistants, but to provide a repository-oriented tool with repeatable indexing and verifiable source references.

Future Scope: Future improvements may include more programming-language support, syntax-aware chunking with Tree-sitter, function and class relationship graphs, cross-file execution-flow tracing, GitHub OAuth and private repository access, incremental re-indexing after commits, line-level citations, code change summaries, test generation, and feedback-based retrieval evaluation.

Project Purpose in One Sentence

To reduce the time developers spend understanding unfamiliar codebases by retrieving relevant source code and generating clear, verifiable explanations in response to natural-language questions.